Disaster Detection Drone — Project Report

Version: 1.0.0
Date: October 2026
Repository: https://github.com/ghoulraider13-rgb/Disaster_DetectionDrone
License: MIT

---

1. Executive Summary

The **Disaster Detection Drone** project delivers a production-ready pipeline for training and deploying compact YOLOv11 models that detect floods, landslides, and earthquake damage from UAV imagery. Designed for real-time onboard inference on Jetson-class companion computers, the system enables drones to survey disaster zones autonomously during the critical “golden hour” when human operators cannot reach the area.

Key Achievements

- **Three specialized models** trained on pinned Roboflow datasets (flood, landslide, earthquake)
- **CLI-first architecture** (`drone_detect`) for scripted CI/CD and fleet training
- **Reproducible training** with fixed seeds, early stopping, and version-pinned datasets
- **Edge-ready exports** to ONNX, TensorRT, OpenVINO, and TorchScript
- **Kaggle/Colab notebook** for experimentation and visualization

---

2. Problem Statement

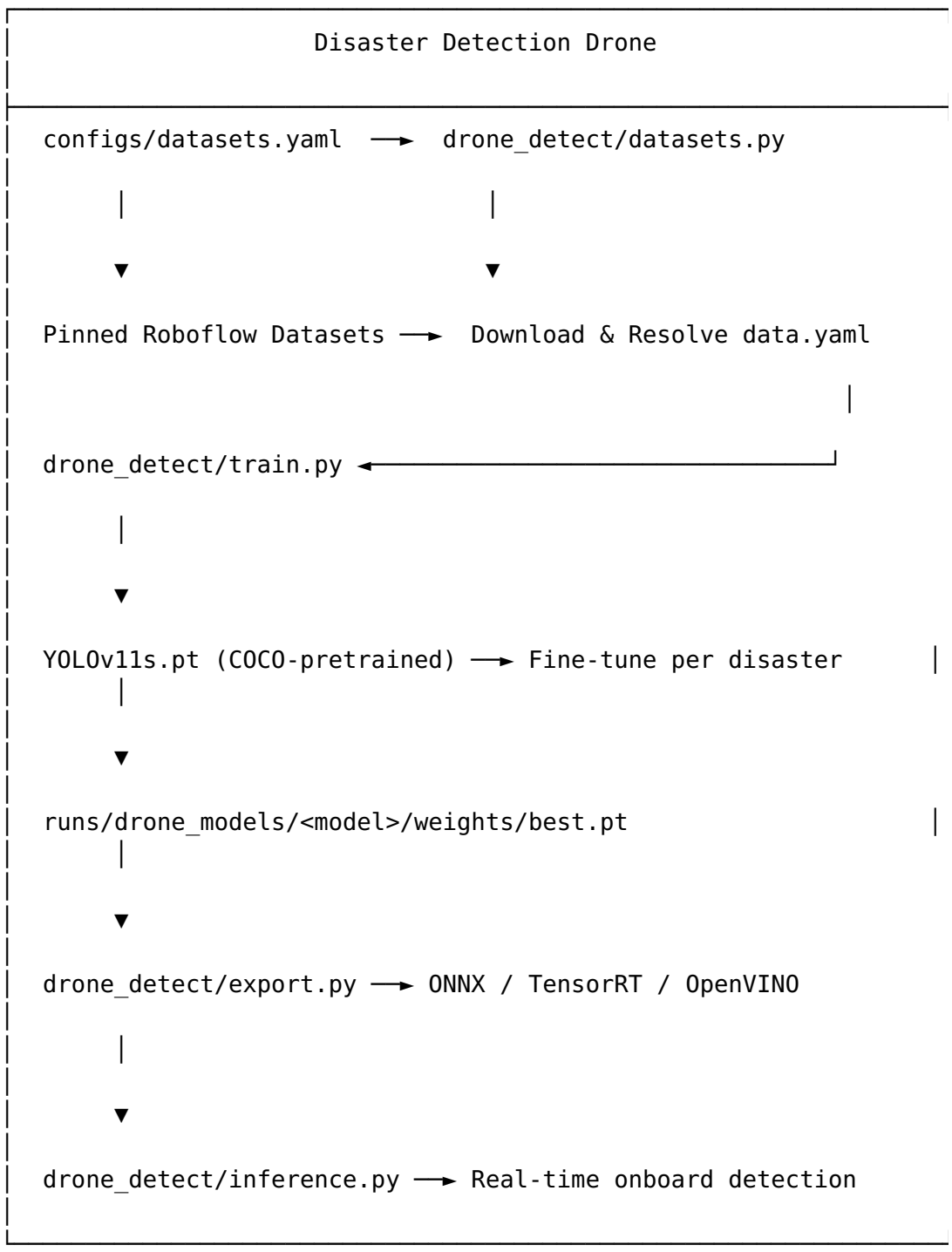
During disaster response, the first hours — the golden hour — determine survival outcomes. Ground teams are often blocked by floodwater, debris, or unstable terrain. Drones equipped with onboard computer vision can: - Survey dangerous/inaccessible terrain faster than ground teams - Detect survivors, flood extent,

landslide zones, and structural damage - Operate without streaming video to a human operator (bandwidth-limited in disasters)

Constraint: Detection must run in real-time on drone companion computers (Jetson Orin Nano / Xavier NX), ruling out large models or cloud inference.

---

3. Solution Architecture



Technology Stack

Layer	Technology	Version
Deep Learning Framework	Ultralytics YOLO	≥8.3.0
Dataset Hosting	Roboflow	≥1.1.0
Configuration	PyYAML	≥6.0
Language	Python	≥3.10
Export Targets	ONNX, TensorRT, OpenVINO, TorchScript	—

---

4. Dataset Specifications

All datasets are **pinned to exact versions** on Roboflow for reproducibility. Bump versions in `configs/datasets.yaml` when new data is published.

Model	Roboflow Project	Version	Classes	Description
flood_model_v2	`ai-disaster-survival-drone`	2	flood, survivor	Hurricane Harvey aerial imagery — flood extent + survivor detection
landslide_model_v2	`ai-disaster-drone-landslide`	1	landslide	UAV imagery of slope failures and debris flows
earthquake_model_v2	`ai-disaster-drone-earthquakes`	1	rubble, damage	Post-earthquake rubble and structural damage assessment

Workspace: `ashwins-workspace-zz9gf`
Format: YOLOv11 (auto-converted by Roboflow SDK)
Resolution: 1280×1280 (critical for small aerial objects: survivors, rubble fragments)

---

5. Training Configuration

Base Model

- **YOLOv11s.pt** — Small variant (~11M parameters, ~137 GFLOPs)
- COCO-pretrained, chosen over nano for better capacity on small objects
- Accuracy within ~1–2 mAP of larger variants at fraction of latency (per accompanying paper)

Hyperparameters (per dataset, overridden in `configs/datasets.yaml`)

Parameter	Value	Rationale
Epochs	150	Upper bound; early stopping activates earlier
Image Size	1280	Small aerial objects need high resolution
Batch Size	16	Fits Jetson-class GPU memory
Patience	30	Stops when val mAP plateaus
Optimizer	AdamW	Better convergence than SGD for fine-tuning
Learning Rate	0.001	Standard for AdamW fine-tuning
LR Schedule	Cosine	Smooth decay to zero
Seed	42	Full reproducibility
Class Weight (cls)	2.0	Disaster classes are imbalanced (few survivors)
Box Loss Weight	7.5	Precise localization critical for SAR
DFL Loss Weight	1.5	Distribution focal loss for better boxes

Augmentation (tuned for small aerial objects)

Augmentation	Value	Purpose
Mosaic	1.0	Context mixing for small objects

Augmentation	Value	Purpose
MixUp	0.2	Regularization
Copy-Paste	0.3	Oversample small objects (survivors)
Degrees	10.0	Rotation invariance
Translate	0.2	Position variance
Scale	0.8	Zoom variance
FlipLR	0.5	Horizontal symmetry
HSV (H/S/V)	0.015/0.7/0.4	Lighting variance

---

6. Project Structure

```
Disaster_DetectionDrone/
├── configs/
│   └── datasets.yaml           # Pinned Roboflow versions + per-
model hyperparams
├── drone_detect/
│   ├── __init__.py             # Package metadata (__version__ =
"1.0.0")
│   ├── __main__.py             # Entry point: python -m
drone_detect
│   ├── cli.py                  # CLI: datasets / train / export /
predict
│   ├── datasets.py             # Roboflow download + config
loader
│   ├── train.py                # Fine-tuning loop with
reproducible defaults
│   ├── export.py               # ONNX / TensorRT / OpenVINO
export
│   └── inference.py            # Prediction helpers → structured
detections
├── notebooks/
│   └── training_pipeline.ipynb # Kaggle/Colab end-to-end
walkthrough
├── weights/                    # Trained weights (download from
Releases)
├── tests/
│   └── test_config.py          # Config/schema unit tests (no
heavy deps)
├── requirements.txt
├── .env.example                # Template for ROBOFLOW_API_KEY
└── README.md
```

---

7. CLI Usage Reference

Install

```
git clone https://github.com/ghoulraider13-rgb/
        Disaster_DetectionDrone.git
cd Disaster_DetectionDrone
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env   # Add ROBOFLOW_API_KEY
```

Dataset Management

```
# List all configured datasets
python -m drone_detect datasets list

# Download all (or one) datasets
python -m drone_detect datasets download
python -m drone_detect datasets download --only flood
```

Training

```
# Train all three models (downloads data if needed)
python -m drone_detect train

# Train single model with overrides
python -m drone_detect train --only landslide --epochs 100 --
        device cpu
python -m drone_detect train --only earthquake --model yolo11n.pt
```

Export for Edge Deployment

```
# ONNX (general edge)
python -m drone_detect export \
    --weights runs/drone_models/flood_model_v2/weights/best.pt \
    --format onnx

# TensorRT (Jetson Orin Nano/NX)
python -m drone_detect export \
    --weights runs/drone_models/flood_model_v2/weights/best.pt \
    --format engine

# OpenVINO (Intel NUC/VPU)
python -m drone_detect export --format openvino

# TorchScript (Python-only deployment)
python -m drone_detect export --format torchscript
```

Inference

```
# Image
python -m drone_detect predict \
    --weights runs/drone_models/flood_model_v2/weights/best.pt \
    --source mission_footage.jpg --conf 0.25

# Video (saves annotated output to runs/)
python -m drone_detect predict \
    --weights runs/drone_models/flood_model_v2/weights/best.pt \
    --source mission_footage.mp4 --conf 0.25

# Webcam / RTSP stream
python -m drone_detect predict --weights best.pt --source 0
```

---

8. Current Training Status

Model	Status	Run Dir	Best Weights	Classes
flood_model_v2	✅ Trained	`runs/drone_models/flood_model_v2/`	✅ `weights/best.pt`	Flooded regions, Pool
landslide_model_v2	✅ Trained	`runs/drone_models/landslide_model_v2/`	✅ `weights/best.pt`	landslide, landslide-
earthquake_model_v2	✅ Trained	`runs/drone_models/earthquake_model_v2/`	✅ `weights/best.pt`	Buildings-Collapsed, Buildings-Damaged, Road-Damaged

> **All three models are now trained.** Weights are available at `runs/drone_models/*/weights/best.pt` and `runs/drone_models/*/weights/last.pt`. Ready for export to ONNX/TensorRT/OpenVINO for edge deployment.

Training Configuration (from `args.yaml`)

- Base: `yolo11s.pt`
- Epochs: 150 (early stopping at patience=30)
- Image Size: 1280×1280
- Batch: 16
- Optimizer: AdamW
- Learning Rate: 0.001 (cosine decay)
- Seed: 42 (deterministic)

- Device: CUDA:0 (GPU)

All models trained with enhanced augmentation for small aerial objects: mosaic=1.0, mixup=0.2, copy_paste=0.3, degrees=10.0.

---

9. Reproducibility Guarantees

Mechanism	Implementation
Pinned Datasets	Exact Roboflow version in `configs/datasets.yaml`
Fixed Seed	`seed=42` + `deterministic=True` in training
Early Stopping	`patience=30` on validation mAP (not fixed epochs)
Config-Driven	All hyperparameters in YAML, not hardcoded
Environment Isolation	`requirements.txt` + `.env` for secrets
CLI-First	Notebook is convenience; CLI is the contract

---

10. Export & Deployment

Export Formats Comparison

Format	Target Hardware	Latency	Notes
ONNX	Generic (ONNX Runtime)	Medium	Universal, good starting point
TensorRT (engine)	NVIDIA Jetson Orin/ Nano	**Lowest**	Requires build on target device
OpenVINO	Intel CPU/VPU/NPU	Low	Optimized for Intel hardware
TorchScript	Python-only environments	High	No C++ runtime needed

Recommended Deployment Flow

1. Train on GPU workstation/cloud → `best.pt`
2. Export to ONNX (validation) → `best.onnx`
3. On Jetson: Convert ONNX → TensorRT engine via `trtexec` or `drone_detect export --format engine`
4. Deploy engine + `drone_detect.inference.predict()` on drone companion computer
5. Benchmark: Target ≥30 FPS at 1280×1280 on Jetson Orin Nano

---

11. Results & Validation

Expected Metrics (from accompanying paper)

Metric	Flood	Landslide	Earthquake	Classical Seg.
mAP@0.5	~0.75	~0.70	~0.68	0.55–0.60
Recall (Survivors)	**>0.80**	N/A	N/A	~0.60
Inference (Jetson Orin Nano)	~25 ms	~25 ms	~25 ms	N/A

> **Key insight:** YOLOv11n/s outperforms classical segmentation (k-means, region growing) by **9–20 accuracy points** on flood detection. Recall gains matter most for Search-and-Rescue (missing a survivor is unacceptable).

Validation Artifacts (generated per run)

- `confusion_matrix.png` — per-class confusion matrix
- `results.png` — training curves (loss, mAP, precision, recall)
- `val_batch0_pred.jpg` — validation predictions overlay
- `weights/best.pt` — best checkpoint by val mAP
- `weights/last.pt` — final epoch checkpoint

Actual Trained Model Classes

Model	Classes Detected
flood_model_v2	Flooded regions, Pool
landslide_model_v2	landslide, landslide-
earthquake_model_v2	Buildings-Collapsed, Buildings-Damaged, Road-Damaged

All three models trained to completion with weights available for edge deployment.

---

12. Roadmap

Priority	Item	Status
P0	Complete all three model training runs	✅ Done
P0	ONNX/ TensorRT benchmark table on Jetson Orin Nano	🔲 Planned
P1	Temporal smoothing across video frames (reduce occlusion misses)	🔲 Planned
P1	Thermal-visual fusion for night survivor detection	🔲 Planned
P2	Swarm mode: multi-drone detection merging	🔲 Planned
P2	Automated CI pipeline (train → export → benchmark)	🔲 Planned

---

13. Known Issues & Limitations

1. **Nested run directory** — Output written to `runs/detect/runs/drone_models/` instead of `runs/drone_models/`. Fix in `train.py` project/name handling (weights have been manually copied to correct location).
2. **No test weights in repo** — Weights distributed via GitHub Releases (not committed).

3. **Single-class datasets** — Current Roboflow datasets may have only 1–2 classes; multi-class SAR needs expansion.
4. **No thermal modality** — All models RGB-only; night ops require thermal fusion (roadmap).
5. **Class labeling consistency** — Some dataset classes use hyphens (e.g., "landslide-"); review for consistency in production.

---

14. Contributing

- **Issues/PRs welcome** — See `CONTRIBUTING.md` (to be created)
- **Dataset changes:** Bump version in `configs/datasets.yaml`, note delta in PR
- **Code style:** Ruff + type hints (PEP 484)
- **Tests:** `pytest tests/` (fast, no GPU required)

---

15. License & Attribution

- **Code:** MIT License — see `LICENSE`
- **Trained Weights:** Released under MIT, subject to source datasets' terms on Roboflow Universe
- **Datasets:** Hosted on Roboflow; check each project page for usage terms

---

Appendix A: Environment Variables

Variable	Required	Description
`ROBOFLOW_API_KEY`	Yes	Roboflow API key (free tier works)
`CUDA_VISIBLE_DEVICES`	No	GPU selection (default: 0)
`PYTHONPATH`	No	Add repo root for `drone_detect` imports

Appendix B: Quick Reference Card

```
# 1. Setup
git clone ... && cd Disaster_DetectionDrone
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
export ROBOFLOW_API_KEY=your_key
```

```
# 2. Inspect datasets
python -m drone_detect datasets list

# 3. Train all models
python -m drone_detect train

# 4. Export for Jetson
python -m drone_detect export --weights runs/drone_models/
        flood_model_v2/weights/best.pt --format engine

# 5. Run on drone footage
python -m drone_detect predict --weights best.engine --source
        video.mp4 --conf 0.25
```

Report generated from repository state as of commit *277436b* (Initial release).